9. To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation.

Code (line 7 - #define MAX_INPUT 200 ➔ #define INPUT_SIZE 200)

ketana_sindhu@KetaZone: ~/

GNU nano 8.7.1

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>

#define MAX_INPUT 200
#define MAX_ARGS 10

char currentDirectory[PATH_MAX];
char previousDirectory[PATH_MAX];

/* Update current working directory */
void updateDirectory() {
    if (getcwd(currentDirectory, sizeof(currentDirectory)) == NULL) {
        perror("getcwd");
    }
}

/* Display current directory */
void builtin_pwd() {
    updateDirectory();
    printf("%s\n", currentDirectory);
}

/* Change directory */
void builtin_cd(char *path) {

    char oldDirectory[PATH_MAX];

    /* Store current directory before changing */
    if (getcwd(oldDirectory, sizeof(oldDirectory)) == NULL) {
        perror("getcwd");
        return;
    }

    /* If no path is given, go to HOME */
    if (path == NULL) {

        path = getenv("HOME");

        if (path == NULL) {
            printf("cd: HOME not set\n");
            return;
        }
    }

    /* Handle previous directory */
    if (strcmp(path, "-") == 0) {

        if (strlen(previousDirectory) == 0) {
            printf("cd: previous directory not available\n");
            return;
        }

        path = previousDirectory;
    }

    /* Validate and change directory */
    if (chdir(path) != 0) {
```

```c
    /* Validate and change directory */
    if (chdir(path) != 0) {

        perror("cd");
        return;
    }

    /* Save previous directory */
    strcpy(previousDirectory, oldDirectory);

    /* Update current directory */
    updateDirectory();

    printf("Current directory: %s\n", currentDirectory);
}

/* Parse user input */
int parseInput(char *input, char *args[]) {

    int count = 0;

    char *token = strtok(input, " \t\n");

    while (token != NULL && count < MAX_ARGS - 1) {

        args[count++] = token;

        token = strtok(NULL, " \t\n");
    }

    args[count] = NULL;

    return count;
}

int main() {

    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    previousDirectory[0] = '\0';

    updateDirectory();

    printf("Directory Navigation Shell\n");
    printf("--------------------------\n");
    printf("Commands: pwd, cd <path>, cd -, exit\n\n");

    while (1) {

        printf("shell[%s]> ", currentDirectory);
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL) {
            break;
        }

        input[strcspn(input, "\n")] = '\0';
```

```c
    while (1) {

        printf("shell[%s]> ", currentDirectory);
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL) {
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        /* Ignore empty input */
        if (strlen(input) == 0) {
            continue;
        }

        parseInput(input, args);

        /* Exit */
        if (strcmp(args[0], "exit") == 0) {
            printf("Exiting shell...\n");
            break;
        }

        /* pwd */
        else if (strcmp(args[0], "pwd") == 0) {
            builtin_pwd();
        }

        /* cd */
        else if (strcmp(args[0], "cd") == 0) {
            builtin_cd(args[1]);
        }

        /* Invalid command */
        else {
            printf("Invalid command: %s\n", args[0]);
        }
    }

    return 0;
}
```

Result:

```
ketana_sindhu@KetaZone: ~/
bheeshma@SINGAM:~/SKILL$ nar                    o Skill9a.c
bheeshma@SINGAM:~/SKILL$ gc                     : Skill9a.c -o s9a
bheeshma@SINGAM:~/SKILL$ ./                     9a
Directory Navigation Shell
--------------------------
Commands: pwd, cd <path>, cd -, exit

shell[/home/ketana_sindhu/SKILL]> pwd
/home/ketana_sindhu/SKILL
shell[/home/ketana_sindhu/SKILL]> cd ..
Current directory: /home/ketana_sindhu
shell[/home/ketana_sindhu]> cd SKILL
Current directory: /home/ketana_sindhu/SKILL
shell[/home/ketana_sindhu/SKILL]> exit
Exiting shell...
bheeshma@SINGAM:~/SKILL$
```

To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic.

Code

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define INPUT_SIZE 200
#define MAX_ARGS 20

/* Function pointer for built-in commands */
typedef int (*BuiltinFunction)(char **args);

/* Maintain shell state */
char current_directory[INPUT_SIZE];

/* ---------- BUILT-IN COMMANDS ---------- */

int builtin_pwd(char **args) {
    (void)args;

    getcwd(current_directory, sizeof(current_directory));
    printf("%s\n", current_directory);

    return 1;
}

int builtin_cd(char **args) {

    if (args[1] == NULL) {
        printf("cd: missing argument\n");
        return 1;
    }

    if (chdir(args[1]) != 0) {
        perror("cd");
        return 1;
    }

    getcwd(current_directory, sizeof(current_directory));

    printf("Directory changed to: %s\n", current_directory);

    return 1;
}

int builtin_echo(char **args) {

    for (int i = 1; args[i] != NULL; i++) {
        printf("%s", args[i]);

        if (args[i + 1] != NULL)
            printf(" ");
    }
```

```
  GNU nano 8.7.1

    for (int i = 1; args[i] != NULL; i++) {
        printf("%s", args[i]);

        if (args[i + 1] != NULL)
            printf(" ");
    }

    printf("\n");

    return 1;
}

int builtin_help(char **args) {
    (void)args;

    printf("\nBuilt-in Commands:\n");
    printf("  pwd              - Display current directory\n");
    printf("  cd <directory>   - Change directory\n");
    printf("  echo <text>      - Display text\n");
    printf("  help             - Display available commands\n");
    printf("  exit             - Exit shell\n");

    return 1;
}

int builtin_exit(char **args) {
    (void)args;

    printf("Exiting shell...\n");

    return 0;
}

/* ---------- DISPATCH TABLE ---------- */

typedef struct {
    char *name;
    BuiltinFunction function;
} Builtin;

Builtin dispatch_table[] = {
    {"pwd",  builtin_pwd},
    {"cd",   builtin_cd},
    {"echo", builtin_echo},
    {"help", builtin_help},
    {"exit", builtin_exit}
};

int builtin_count =
    sizeof(dispatch_table) / sizeof(dispatch_table[0]);
```

```c
/* ---------- DISPATCH TABLE ---------- */

typedef struct {
    char *name;
    BuiltinFunction function;
} Builtin;

Builtin dispatch_table[] = {
    {"pwd",  builtin_pwd},
    {"cd",   builtin_cd},
    {"echo", builtin_echo},
    {"help", builtin_help},
    {"exit", builtin_exit}
};

int builtin_count =
    sizeof(dispatch_table) / sizeof(dispatch_table[0]);

/* ---------- PARSE INPUT ---------- */

int parse_input(char *input, char *args[]) {

    int count = 0;

    char *token = strtok(input, " \t\n");

    while (token != NULL && count < MAX_ARGS - 1) {

        args[count++] = token;

        token = strtok(NULL, " \t\n");
    }

    args[count] = NULL;

    return count;
}

/* ---------- DISPATCH COMMAND ---------- */

int execute_builtin(char **args) {

    if (args[0] == NULL)
        return 1;

    for (int i = 0; i < builtin_count; i++) {

        if (strcmp(args[0], dispatch_table[i].name) == 0) {

            return dispatch_table[i].function(args);
```



```
  GNU nano 8.7.1
/* ---------- DISPATCH COMMAND ---------- */

int execute_builtin(char **args) {

    if (args[0] == NULL)
        return 1;

    for (int i = 0; i < builtin_count; i++) {

        if (strcmp(args[0], dispatch_table[i].name) == 0) {

            return dispatch_table[i].function(args);
        }
    }

    printf("Invalid command: %s\n", args[0]);
    printf("Type 'help' to see available commands.\n");

    return 1;
}

/* ---------- MAIN ---------- */

int main() {

    char input[INPUT_SIZE];
    char *args[MAX_ARGS];

    getcwd(current_directory, sizeof(current_directory));

    printf("Built-in Command Shell\n");
    printf("----------------------\n");
    printf("Type 'help' for available commands.\n\n");

    while (1) {

        printf("shell[%s]> ", current_directory);
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0)
            continue;

        parse_input(input, args);

        int result = execute_builtin(args);

        if (result == 0)
```

```c
/* ---------- MAIN ---------- */

int main() {

    char input[INPUT_SIZE];
    char *args[MAX_ARGS];

    getcwd(current_directory, sizeof(current_directory));

    printf("Built-in Command Shell\n");
    printf("----------------------\n");
    printf("Type 'help' for available commands.\n\n");

    while (1) {

        printf("shell[%s]> ", current_directory);
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0)
            continue;

        parse_input(input, args);

        int result = execute_builtin(args);

        if (result == 0)
            break;
    }

    return 0;
}
```

Result:

```
ketana_sindhu@KetaZone: ~/

bheeshma@SINGAM:~/SKILL$ nano Sk          ill9b.c
bheeshma@SINGAM:~/SKILL$ gcc Ski          ll9b.c -o s9b
bheeshma@SINGAM:~/SKILL$ ./s9b
Built-in Command Shell
----------------------
Type 'help' for available commands.

shell[/home/ketana_sindhu/SKILL]> pwd
/home/ketana_sindhu/SKILL
shell[/home/ketana_sindhu/SKILL]> echo Hello World
Hello World
shell[/home/ketana_sindhu/SKILL]> help

Built-in Commands:
  pwd             - Display current directory
  cd <directory>  - Change directory
  echo <text>     - Display text
  help            - Display available commands
  exit            - Exit shell
shell[/home/ketana_sindhu/SKILL]> cd ..
Directory changed to: /home/ketana_sindhu
shell[/home/ketana_sindhu]> pwd
/home/ketana_sindhu
shell[/home/ketana_sindhu]> invalid
Invalid command: invalid
Type 'help' to see available commands.
shell[/home/ketana_sindhu]> exit
Exiting shell...
bheeshma@SINGAM:~/SKILL$
```